

# CURSO DE CAPACITAÇÃO MYSQL

Sistema de Gerenciamento de Banco de Dados Relacional (SGBDR)

*Módulo Básico ao Intermediário*

**Instrutor:** José Charles Curvina de Menezes  
Auxiliar Tecnológico / Técnico de Informática

**Carga Horária:** 40 horas  
**Local:** Laboratório de Informática / J2CM  
**Data:** Agosto de 2025

## ROTEIRO DO CURSO DE MYSQL – J2CM

- 1° - CRIE UM REPOSITÓRIO NO GITHUB.COM
- 2° - COPIAR A URL PARA CRIAÇÃO DO REPOSITÓRIO LOCAL
  - CÓDIGO
  - Copiar a URL para criação do Repositório LOCAL
  - <https://github.com/JCharlesCM-hub/Cursos-SQL.git>
- 3° - ABRIR A PASTA DO REPOSITÓRIO LOCAL COM O VISUAL CODE – VSCODE
  - Vá até a pasta criada para o repositório local com o Windows Explorer
  - Selecione a barra de endereço e digite \$ CMD
  - Depois digite \$ CODE . (abrir o VSCODE)
- 4° - AO ABRIR O VSCODE
  - Clicar em Terminal / Novo Terminal
  - Ao abrir o terminal no lado direito na seta(+V) para abrir o Terminal bash
  - Clique em bash e abrirá o terminal \$
  - \$ git clone <https://github.com/jcharlescm/Curso-SQL.git>
- 4° - INSTALAR O MYSQL
  - No GOOGLE pesquisar por MYSQL
  - <https://www.mysql.com/downloads/>
    - Downloads da Comunidade MySQL (GPL) »
    - Instalador MySQL para Windows
    - Windows (x86, 32 bits), instalador MSI(350 MB)
  - Não fazer login:  
Não, obrigado, apenas inicie meu download.
- 5° - Baixa e copiar para:  
F:\MeusArq\Cursos\Curso\_SQL\Banco-MYSQL
  - Tutorial de MYSQL
  - <https://www.devmedia.com.br/mysql-tutorial/33309>
- 6° - Executar o arquivo e next, next ...
  - Instalar a versão completa
  - MySQL Server, MySQL Workbench, MySQL Shell ...
  - Senha: j2cm1928
  - Next, next... informar a senha novamente e checar, next..
  - Clicar em Local Instance MySQL80 e informar a senha...
- 7° - Abrir o MySQL Workbench
  - Create new schema, no ícone com + e o Disco
  - Em base schema
  - criar um schema: base.
- 8° - Importar as tabelas F:\MeusArq\Cursos\Curso\_SQL\Banco-MYSQL\Base
  - Clicar em Server/Data Import --> F:\MeusArq\Cursos\Curso\_SQL\Banco-MYSQL
  - Clicar em Start Import
- 9° - Serão importadas as tabelas: categorias; clientes; locais; pedidos; produtos;
  - Pode fechar a aba de DATA IMPORT
- 10° - Clicar com o botão direito sobre BD base / Set default schema: será definido o BD base como default para as consultas query, ficará em negrito.

```

/*
34 - MYSQL - Consultas Básicas

1. Antes de começar, alguns comentários...
*/
# TIPOS DE COMENTÁRIOS

# Este é um Comentario de linha

-- Este é outro Comentário de linha


/*
Este é um comentario de Bloco
PARTE 01 COMENTARIO
PARTE 02 COMENTARIO
PARTE 03 COMENTARIO
PARTE 04 COMENTARIO
*/

-- 2. SELECT (Parte 1)
Select * From base.categorias;
Select * From base.clientes;
Select * From base.locais;
Select * From pedidos;
Select * From produtos;

# Montar uma instrução SQL automática: Clicar com o botão direito sobre a tabela /
Select Rows.
-- Será criado outra aba de query.alter

-- Ctrl + botão do meio --> Almentar ou diminui a fonte da query

-- Selecionar todas as colunas:
Select * From base.pedidos;
Select * From base.produtos;

-- Sem a especificação do BD base:
Select * From pedidos;
Select * From produtos;

-- Selecionar apenas uma coluna
Select Nome_Produto From produtos;
Select produtos.Nome_Produto From base.produtos;

-- 3. E se eu não definir um banco de dados como padrão
Select base.produtos.Nome_Produto From base.produtos;
Select base.produtos.* From base.produtos;

-- 4. Formas de executar os códigos no MySQL
-- 5. Cuidados com o ponto e vírgula

-- Selecionar varias coluna específicas
Select ID_Produto, Nome_Produto, Marca_Produto, Num_Serie From produtos;

-- 6. SELECT (Parte 2)

-- Mesma SQL de cima, melhores praticas
Select
    ID_Produto,
    Nome_Produto,
    Marca_Produto,
    Num_Serie
From produtos;

```

```

# Comando AS: Renomeando as colunas
Select
    ID_Produto AS 'ID_Produto',
    Nome_Produto AS 'Nome do produto',
    Marca_Produto AS 'Marca do produto',
    Custo_Unit AS 'Custo do produto'
From produtos;

-- 7. AS - Renomeando Colunas e Tabela
# Comando AS: Renomeando a Tabela
Select
    p.ID_Produto,
    p.Nome_Produto,
    p.Marca_Produto,
    p.Num_Serie
From produtos AS p;

-- 8. LIMIT e OFFSET
# LIMIT: Limitar o numero de linhas
Select * From produtos LIMIT 5;
Select * From pedidos LIMIT 10;

# LIMIT + OFFSET: Indica o inicio da Leitura
Select * From produtos LIMIT 5 OFFSET 5;

-- 9. DISTINCT
-- Seleciona valores distintos da coluna
Select * From produtos;
Select DISTINCT Marca_Produto From produtos;

-- 10. ORDER BY
# Ordenar coluna de forma DESC
Select *
From clientes
ORDER BY Nome ASC;

Select *
From clientes
ORDER BY Nome, Sobrenome;

Select *
From clientes
ORDER BY Renda_Anual;

Select *
From clientes
ORDER BY Data_Nascimento;

# Ordenar coluna de forma DESC
Select *
From clientes
ORDER BY Nome DESC;

Select *
From clientes
ORDER BY Renda_Anual DESC;

Select *
From clientes
ORDER BY Data_Nascimento DESC;

Select *
From clientes
ORDER BY Renda_Anual DESC, Data_Nascimento DESC;

```

### # 35. [MySQL] Filtros no MySQL

#### -- 1. Comando WHERE

# O comando WHERE pode ser usado em conjunto com os operadores AND e

-- OR para filtrar mais de uma coluna ao mesmo tempo e também com o

-- operador NOT, para criar negociações.

# WHERE: Crie um filtro na tabela de clientes para mostrar apenas os clientes com renda anual >= 8000

```
SELECT *
FROM Clientes
WHERE Renda_Anual >= 8000;
```

# WHERE: Crie um filtro na tabela de clientes para mostrar apenas os clientes com sexo igual Masculino.

```
SELECT *
FROM Clientes
WHERE Sexo = 'M';
```

```
Select *
From clientes
WHERE Data_Nascimento > '2000-01-01'; # formato 'AAAA-MM-DD';
```

#### # 2. Operadores AND, OR e NOT

-- (AND)

-- SELECT coluna1, coluna2, coluna3

-- FROM tabela

-- WHERE Sexo = 'F' AND Escolaridade = 'Parcial' AND Coluna2 = valor3;

```
Select *
From produtos
WHERE Marca_Produto = 'DELL' AND Preco_Unit >= 2000;
```

-- (OR)

-- SELECT coluna1, coluna2, coluna3

-- FROM tabela

-- WHERE Marca\_Produto = 'DELL' OR Marca\_Produto = 'SANSUNG';

```
Select *
From produtos
WHERE Marca_Produto = 'DELL' OR Marca_Produto = 'ALTURA';
```

-- (NOT)

-- SELECT coluna1, coluna2, coluna3

-- FROM tabela

-- WHERE NOT Marca\_Produto = 'DELL';

-- outra forma: WHERE NOT Marca\_Produto <> 'DELL';

```
Select *
From produtos
WHERE NOT Marca_Produto = 'SAMSUNG';
```

-- ou dessa forma

```
Select *
From produtos
WHERE Marca_Produto <> 'SAMSUNG';
```

#### # 3. IS NULL e IS NOT NULL

```
Select *
From Lojas
WHERE Telefone IS NOT NULL;
-- Campo em branco não é NULL
```

```
Select *
From clientes
WHERE Telefone = '';
-- juntando registros nulos e em branco
```

```
Select *
From clientes
WHERE Telefone IS NULL OR Telefone = '';
-- juntando todos os Não Nulos e os Vazios...
```

```
Select *
From clientes
WHERE Telefone IS NOT NULL OR Telefone = '';
```

#### # 5. WHERE e LIKE

# O operador LIKE no MySQL é usado em conjunto com a cláusula WHERE para realizar buscas parciais em campos de texto de uma tabela, utilizando caracteres curinga para especificar o padrão de pesquisa.

```
SELECT *
FROM clientes
WHERE Email LIKE 'gmail';
```

```
SELECT *
FROM clientes
WHERE Email LIKE '%gmail%';
```

```
SELECT *
FROM clientes
WHERE Email LIKE '%.br';
```

#### # 6. WHERE e IN, NOT IN

# Os comandos WHERE e IN/NOT IN são usados para filtrar dados em consultas. WHERE define a condição para seleção dos registros, e IN/NOT IN são operadores que permitem verificar se um valor está presente ou não em uma lista de valores.

```
Select *
From produtos
WHERE Marca_Produto = 'DELL' OR Marca_Produto = 'SONY' OR Marca_Produto = 'ALTURA';
```

```
Select *
From produtos
WHERE Marca_Produto IN('DELL', 'SONY', 'ALTURA');
```

#### # 7. WHERE e BETWEEN

```
SELECT *
FROM produtos
WHERE Preco_Unit BETWEEN 1000 AND 2500;
```

```
SELECT *
FROM clientes
WHERE Data_Nascimento BETWEEN '1995-01-01' AND '1999-12-31';
```

### # 36. [MySQL] Operações Básicas e Funções de Agregação

#### # 1. Operações básicas e funções de arredondamento

```
SELECT
    10 + 20          AS 'Soma',
    100 - 40         AS 'Subtração',
    5 * 20           AS 'Multiplicação',
    300 / 12         AS 'Divisão',
    (100 - 10) * 4 AS 'Operação',
    22 % 5           AS 'Resto da Divisão';
```

#### # Funções de Arredondamento

```
SELECT
    ROUND(87.149, 2)      AS 'Arred.',
    FLOOR(87.149)         AS 'Arred. p/ baixo',
    CEILING(87.149)       AS 'Arred. p/ cima',
    TRUNCATE(87.149, 2)   AS 'Truncar';
```

#### # 2. COUNT, COUNT DISTINCT

# Em MySQL, COUNT e COUNT(DISTINCT) são funções de agregação usadas para contar registros em uma tabela. COUNT(\*) conta todas as linhas, enquanto COUNT(DISTINCT coluna) conta apenas os valores únicos em uma coluna específica, excluindo valores NULL.

```

# COUNT(*):
-- Conta todas as linhas em uma tabela ou conjunto de resultados, incluindo linhas
duplicadas e valores NULL.
-- Exemplo: SELECT COUNT(*) FROM usuarios; conta todas as linhas na tabela "usuarios".
SELECT *
FROM clientes;
-- Conta todas as linhas
SELECT
    COUNT(*)
FROM clientes;
-- Conta todos os nomes
SELECT
    COUNT(Nome) AS 'Qtd. Clientes'
FROM clientes;

# COUNT(DISTINCT):
-- Conte a quantidade de marcas distintas na tabela de PRODUTOS
SELECT
FROM produtos;
-- Selecciona a Quantidade
Select
    COUNT(DISTINCT Marca_Produto)
FROM produtos;
-- Selecciona as marcas distintas
Select
    DISTINCT Marca_Produto
FROM produtos;

# 3. SUM, AVG, MIN e MAX
# Em MySQL, as funções de agregação SUM, AVG, MIN e MAX são usadas para realizar
cálculos em um conjunto de dados.
-- Elas retornam um único valor que resume os valores de uma coluna.
-- SUM calcula a soma, AVG calcula a média,
-- MIN encontra o valor mínimo e
-- MAX encontra o valor máximo.
SELECT *
FROM pedidos;

SELECT
    SUM(Receita_Venda) AS 'Soma de Receita',
    AVG(Receita_Venda) AS 'Média de Receita',
    MIN(Receita_Venda) AS 'Menor Receita',
    MAX(Receita_Venda) AS 'Maior Receita'
FROM pedidos;

# 37. [MySQL] Agrupamentos no MySQL
# 1. GROUP BY (Parte 1)
# O comando GROUP BY em MySQL é utilizado para agrupar linhas com valores iguais em um
conjunto de resultados, geralmente em combinação com funções de agregação como
COUNT(), SUM(), AVG(), MAX() e MIN().
-- Ele permite resumir dados e gerar relatórios baseados em grupos de linhas,
facilitando a análise de dados.
-- Sintaxe:
/*
SELECT column1, column2, ... aggregate_function(column_to_aggregate)
FROM table_name
WHERE condition
GROUP BY column1, column2, ...
ORDER BY column1, column2, ...;
*/
SELECT * FROM produtos;

SELECT Marca_Produto, COUNT(*) AS 'Qtd. de Produtos'
FROM produtos
GROUP BY Marca_Produto;

```

# Crie um agrupamento mostrando o total de clientes por sexo.

```
SELECT * FROM clientes;
```

```
SELECT Sexo, COUNT(*) AS 'Qtd. de Clientes'
FROM clientes
GROUP BY Sexo;
```

# Crie um agrupamento mostrando o total de receita por ID\_Produto.

```
SELECT * FROM pedidos;
```

```
SELECT ID_Produto, COUNT(Receita_Venda) AS 'Total Receita'
FROM pedidos
GROUP BY ID_Produto;
```

# Crie um agrupamento mostrando o total de clientes por escolaridade e sexo.

```
SELECT * FROM clientes;
```

```
SELECT Sexo, Escolaridade, COUNT(*) AS 'Qtd. de Clientes'
FROM clientes
GROUP BY Sexo, Escolaridade;
```

# Crie um agrupamento mostrando o total de receita por ID\_Produto.

```
SELECT * FROM pedidos;
```

```
SELECT ID_Produto, ID_Loja, COUNT(Receita_Venda) AS 'Total Receita'
FROM pedidos
GROUP BY ID_Produto, ID_Loja;
```

# Crie um agrupamento mostrando o total de clientes por escolaridade e sexo feminino.

```
SELECT * FROM clientes;
```

```
SELECT Sexo, Escolaridade, COUNT(*) AS 'Qtd. de Clientes'
FROM clientes
WHERE Sexo = 'F'
GROUP BY Escolaridade;
```

# 4. GROUP BY e HAVING

# A cláusula HAVING é usada para filtrar esses grupos com base em uma condição específica, semelhante à

-- função do WHERE para linhas individuais, mas aplicada a resultados agregados.  
/\*

Neste exemplo:

GROUP BY cidade agrupa os pedidos por cidade.

SELECT cidade, COUNT(\*) AS total\_pedidos seleciona a cidade e o número de pedidos (renomeado como total\_pedidos).

HAVING COUNT(\*) > 100 filtra os grupos, mostrando apenas as cidades com mais de 100 pedidos.

Diferenças entre WHERE e HAVING:

WHERE: Filtra linhas individuais antes que elas sejam agrupadas pelo GROUP BY.

HAVING: Filtra grupos de linhas após o agrupamento pelo GROUP BY.

WHERE não pode ser usado com funções de agregação diretamente. Você precisa usar HAVING para isso.

\*/

```
SELECT * FROM clientes;
```

```
SELECT Escolaridade, COUNT(*) AS 'Qtd. de Clientes'
FROM clientes
GROUP BY Escolaridade
HAVING COUNT(*) > 25;
```

# Produtos com receita total superior a R\$ 5 milhões

```
SELECT ID_Produto, COUNT(Receita_Venda) AS 'Receita Total'
FROM pedidos
GROUP BY ID_Produto
```

```
HAVING SUM(Receita_Venda) >= 5000000;
```

## # 38. [MySQL] Variáveis no MySQL

### # 1. Introdução

# No MySQL, variáveis são usadas para armazenar dados temporários durante a execução de um script ou procedimento armazenado. Elas podem ser declaradas pelo usuário ou serem variáveis de sessão e permitem armazenar valores que podem ser reutilizados em instruções SQL subsequentes.

/\*

Declaração e Atribuição de Variáveis de Usuário:

#### 1. Declaração:

Variáveis de usuário não precisam ser explicitamente declaradas antes de serem usadas. Elas são criadas no momento em que recebem um valor.

#### 2. Atribuição:

Pode-se usar SET @nome\_variavel = valor; ou SET @nome\_variavel := valor; para atribuir um valor à variável.

É possível atribuir valores a variáveis usando SELECT ... INTO @variavel;.

Exemplo:

```
SET @nome_variavel = 10;  
SELECT @outro_nome := campo_tabela FROM tabela WHERE condicao;
```

### Exemplos Práticos:

-- Declaração e atribuição de uma variável de usuário

```
SET @contador = 1;
```

-- Utilizando a variável em uma consulta

```
SELECT * FROM produtos WHERE id = @contador;
```

-- Alterando o valor da variável

```
SET @contador = @contador + 1;
```

-- Verificando o valor da variável

```
SELECT @contador;
```

-- Variáveis de sistema

```
SELECT @@version; -- Mostra a versão do MySQL
```

\*/

# Declaração de variáveis de quantidade

```
SET @varQuant = 10;
```

```
SET @varPreco = 10.9;
```

```
-- SELECT 10 * 10.9 AS 'Receita Total 01';
```

```
-- SELECT @varQuant * @varPreco AS 'Receita Total 02';
```

```
SET @varReceita = @varQuant * @varPreco;
```

```
SELECT @varReceita AS 'Receita Total 03';
```

# Valor armazenado em uma variável

```
SET @varMarca = 'SONY';
```

```
SELECT *
```

```
FROM produtos
```

```
WHERE Marca_Produto = @varMarca;
```

/\*

### # 3. Função CAST

-- A função CAST no MySQL é usada para converter um valor de um tipo de dado para outro.

Sintaxe: CAST(expressão AS tipo\_de\_dado)

Onde:

expressão: É o valor que você deseja converter. Pode ser uma coluna de tabela, uma variável, um valor literal, etc.

tipo\_de\_dado: É o tipo de dado para o qual você deseja converter a expressão. Alguns exemplos comuns são SIGNED, UNSIGNED, CHAR, DATE, DATETIME, DECIMAL, INT, etc.

\*/

```
SET @varNumero = 10.9200;
```

```

SELECT @varNumero,
       CAST(@varNumero AS SIGNED),
       CAST(@varNumero AS DECIMAL(10, 2)),
       CAST(@varNumero AS CHAR(3));

```

# Exemplo com data

```
SET @varData = '2021-01-28';
```

```

SELECT @varData,
       CAST(@varData AS DATE),
       CAST(@varData AS DATETIME);

```

# 39. [MySQL] Funções de Texto e Data no MySQL

# 1. Função LENGTH

-- Em MySQL, a função LENGTH() é usada para obter o comprimento de uma string em bytes.

/\*

Sintaxe:

```
LENGTH(string)
```

Parâmetros:

-- string: A string da qual se deseja obter o comprimento.

Retorno:

Um valor inteiro representando o número de bytes na string.

Exemplos:

```
SELECT LENGTH('Olá, Mundo!'); -- Retorna 12 (considerando que 'Olá, Mundo!' tem 12 bytes)
```

```
SELECT nome, LENGTH(nome) AS tamanho_nome FROM usuarios;
```

\*/

```
SET @varCurso = 'SQL Impressionador';
```

```
SELECT LENGTH(@varCurso); -- Escreve o tamanho da string
```

# A quantidade de caracteres de cada nome da tabela clientes

```
SELECT * FROM clientes;
```

```
SELECT
```

```
    Nome AS 'Nome',
```

```
    LENGTH(Nome) AS 'Número de Caract.'
```

```
FROM clientes;
```

# 2. Funções CONCAT e CONCAT\_WS (Parte 1)

# Escrever o nome completo

```
SET @varNome = 'José Charles';
```

```
SET @varSobrenome = 'Curvina';
```

```
SET @varUltimoNome = 'de Menezes';
```

```
SET @varNomeCompleto = CONCAT(@varNome, ' ', @varSobrenome, ' ', @varUltimoNome);
```

```
SELECT @varNomeCompleto;
```

# 2. Funções CONCAT e CONCAT\_WS (Parte 1)

# Escrever o nome completo

```
SET @varNome02 = 'José Charles';
```

```
SET @varSobrenome02 = 'Curvina';
```

```
SET @varUltimoNome02 = 'de Menezes';
```

```
SET @varNomeCompleto02 = CONCAT_WS(' ', @varNome02, @varSobrenome02, @varUltimoNome02);
```

```
SELECT @varNomeCompleto02;
```

# 3. Funções CONCAT e CONCAT\_WS (Parte 2)

# Criar uma coluna de nome completo

```
SELECT * FROM clientes;
```

```
SELECT
```

```
    ID_Cliente,
```

```
    Nome,
```

```

        Sobrenome,
        CONCAT(Nome, ' ', Sobrenome) AS 'Nome completo 01',
        CONCAT_WS(' ', Nome, Sobrenome) AS 'Nome completo 02'
FROM clientes;

```

#### # 4. Funções LCASE e UCASE

# Em MySQL, as funções LCASE e UCASE são usadas para converter strings em minúsculas e maiúsculas, respectivamente. LCASE é sinônimo de LOWER e UCASE é sinônimo de UPPER.

/\*

Função LCASE (ou LOWER):

Converte todos os caracteres de uma string para minúsculas.

-- É útil para comparações sem distinção entre maiúsculas e minúsculas e para normalização de dados, onde você quer garantir que todas as strings estejam em minúsculas para comparação ou armazenamento.

-- Exemplo: SELECT LCASE('Olá Mundo'); retornaria 'olá mundo'.

Função UCASE (ou UPPER):

-- Converte todos os caracteres de uma string para maiúsculas.

-- Também é usada para comparação sem distinção entre maiúsculas e minúsculas e para normalização de dados.

-- Exemplo: SELECT UCASE('Olá Mundo'); retornaria 'OLÁ MUNDO'.

\*/

```
SELECT LCASE('TestE'); -- Retorna 'teste'
```

```
SELECT UCASE('TestE'); -- Retorna 'TESTE'
```

```
SELECT * FROM clientes WHERE UCASE(nome) = 'RUBEN'; -- Busca por nome sem distinção entre maiúsculas e minúsculas
```

#### # 4. Funções LCASE e UCASE

```
SELECT
```

```
    LCASE(CONCAT(Nome, ' ', Sobrenome)) AS 'Nome completo 01 (CONCAT)',
```

```
    UCASE(CONCAT_WS(' ', Nome, Sobrenome)) AS 'Nome completo 02 (CONCAT_WS)'
```

```
FROM clientes;
```

#### # 4. Funções LCASE e UCASE

# Em MySQL, as funções LEFT() e RIGHT() são utilizadas para extrair substrings de uma string, a partir da esquerda e da direita, respectivamente.

/\*

Função LEFT():

A função LEFT() retorna uma substring especificada do lado esquerdo de uma string. Ela recebe dois argumentos: a string original e o número de caracteres a serem extraídos a partir da esquerda.

Sintaxe: LEFT(string, number\_of\_characters)

Exemplo: SELECT LEFT('MySQL Function', 5); -- Retorna 'MySQL'

Função RIGHT():

A função RIGHT() retorna uma substring especificada do lado direito de uma string. Ela também recebe dois argumentos: a string original e o número de caracteres a serem extraídos a partir da direita.

Sintaxe:

RIGHT(string, number\_of\_characters)

Exemplo: SELECT RIGHT('MySQL Function', 8); -- Retorna 'Function'

\*/

```
SELECT * FROM produtos;
```

```
SELECT * FROM produtos;
```

```
SELECT
```

```
    Num_serie,
```

```
    LEFT(Num_Serie, 6) AS 'Cod 1', -- 6 dígitos da esquerda
```

```
    RIGHT(Num_Serie, 6) AS 'Cod 2' -- 6 dígitos da direita
```

```
FROM produtos;
```

#### # 6. Funções REPLACE

# A função REPLACE no MySQL substitui todas as ocorrências de uma substring em uma string por outra substring. É uma função de manipulação de strings que pode ser usada para modificar o conteúdo de uma string substituindo partes dela.

/\*

Sintaxe:

REPLACE(str, find\_string, replace\_string)

Onde:

str: A string na qual a substituição será feita.  
find\_string: A substring que será substituída.  
replace\_string: A substring que substituirá a find\_string.

Exemplo:

```
SELECT REPLACE('Olá, mundo!', 'mundo', 'João');  
Este comando retornará a string "Olá, João!".
```

\*/

# Substituir um texto por outro texto

```
SET @varTexto = 'HYMYM é a melhor série de comédia.';  
SET @varTextoNovo = REPLACE(@varTexto, 'HYMYM', 'Friends');  
SELECT @varTextoNovo;
```

# Substitua S por Solteiro

```
SELECT * FROM clientes;  
SELECT  
    Nome,  
    Estado_Civil,  
    REPLACE(REPLACE(Estado_Civil, 'S', 'Solteiro'), 'C', 'Casado')  
FROM clientes;
```

# 7. Funções INSTR e MID (Parte 1)

# Em MySQL, as funções INSTR e MID são usadas para manipulação de strings. INSTR retorna a posição da primeira ocorrência de uma substring em outra string, enquanto MID (ou SUBSTRING) extrai uma substring de uma string.

/\*

Função INSTR:

-- A função INSTR(string, substring) retorna a posição inicial (1-indexada) da primeira ocorrência da substring dentro da string. Se a substring não for encontrada, a função retorna 0.

-- Exemplo: SELECT INSTR('Olá, mundo!', 'mundo'); retorna 7.

Função MID (ou SUBSTRING):

-- A função MID(string, start, length) extrai uma substring de string, começando na posição start (1-indexada) e com um comprimento de length caracteres.

-- A função SUBSTRING(string, start, length) é um sinônimo de MID no MySQL.

-- Exemplo: SELECT MID('Olá, mundo!', 7, 5); retorna 'mundo'.

\*/

SELECT

```
    INSTR('Este é um exemplo', 'é') AS pos_e,  
    MID('Este é um exemplo', 6, 2) AS substring_ex,  
    SUBSTRING('Este é um exemplo', 10, 4) AS substring_ex2;
```

# Retorna a posição do caractere '@'.

# Retornar o ID do Email.

-- a.)

```
SET @email = 'jCharlescm@Gamil.com';  
SET @varPosArroba = INSTR(@email, '@');  
SELECT @varPosArroba;
```

-- b.)

```
SET @email2 = 'jcharlescm@Gamil.com';  
SET @varPosArroba2 = INSTR(@email2, '@');  
SET @varIdEmail = MID(@email2, 1, 10);  
SELECT @varIdEmail;
```

-- b.1) Melhor e dinamico

```
SET @email2 = 'jcharlescm@Gamil.com';  
SET @varPosArroba2 = INSTR(@email2, '@');  
SET @varIdEmail = MID(@email2, 1, @varPosArroba2 - 1);  
SELECT @varIdEmail;
```

# Retornar os IDs de todos os registros da Tabela Clientes

```
SELECT * FROM clientes;  
SELECT INSTR(Email, '@') FROM clientes; -- Posição do arroba  
SELECT  
    Email,  
    MID(Email, 1, INSTR(Email, '@') - 1)
```

```
FROM clientes;
```

### # 9. Funções DAY, YEAR e MONTH

# Em MySQL, as funções DAY, MONTH, e YEAR são usadas para extrair partes específicas de uma data. A função DAY retorna o dia do mês (1 a 31) de uma data. A função MONTH retorna o mês (1 a 12) e a função YEAR retorna o ano de uma data.

-- Exemplos:

```
SELECT DAY('2024-07-23'); -- Retorna 23
SELECT MONTH('2024-07-23'); -- Retorna 7
SELECT YEAR('2024-07-23'); -- Retorna 2024
```

```
SELECT * FROM pedidos;
SELECT Data_venda,
       DAY(Data_venda),
       MONTH(Data_venda),
       YEAR(Data_venda)
FROM pedidos;
```

```
SELECT * FROM clientes;
SELECT
       Data_Nascimento,
       DAY(Data_Nascimento) AS 'Dia',
       MONTH(Data_Nascimento) AS 'Mês',
       YEAR(Data_Nascimento) AS 'Ano'
FROM clientes;
```

### # 10. Funções NOW, CURDATE e CURTIME

# Em MySQL, as funções NOW(), CURDATE() e CURTIME() são usadas para trabalhar com datas e horas. NOW() retorna a data e hora atuais. CURDATE() retorna apenas a data atual, enquanto CURTIME() retorna apenas a hora atual.

/\*

NOW():

Retorna a data e hora atuais do sistema do servidor MySQL.

É útil para inserir ou atualizar registros com a data e hora atuais, ou para rastrear o momento em que uma ação foi realizada.

Sintaxe: NOW()

CURDATE():

Retorna a data atual no formato 'YYYY-MM-DD'.

Pode ser usada para obter a data atual em consultas SQL, como ao inserir ou atualizar valores de data ou ao filtrar registros com base na data.

É sinônimo de CURRENT\_DATE().

Sintaxe: CURDATE() ou CURRENT\_DATE()

CURTIME():

Retorna a hora atual no formato 'HH:MM:SS'.

Pode ser usada para obter a hora atual em consultas SQL, como ao inserir ou atualizar valores de tempo.

É sinônimo de CURRENT\_TIME().

Sintaxe: CURTIME() ou CURRENT\_TIME()

\*/

```
SELECT NOW(), CURDATE(), CURTIME();
```

### # 11. Funções DATE\_DIFF, DATE\_ADD e DATE\_SUB

# Em MySQL, as funções DATE\_DIFF, DATE\_ADD e DATE\_SUB são usadas para manipular datas. DATE\_DIFF calcula a diferença entre duas datas, DATE\_ADD adiciona um intervalo a uma data e DATE\_SUB subtrai um intervalo de uma data.

/\*

1. DATE\_DIFF(data1, data2):

Retorna a diferença em dias entre duas datas.

Exemplo: SELECT DATE\_DIFF('2023-10-26', '2023-10-20'); retornará 6.

2. DATE\_ADD(data, INTERVAL valor unidade):

Adiciona um intervalo específico a uma data.

data: A data base.

INTERVAL: Palavra-chave obrigatória.

valor: O valor numérico do intervalo.  
unidade: A unidade de tempo (ex: DAY, MONTH, YEAR, HOUR, MINUTE, SECOND).  
Exemplo: SELECT DATE\_ADD('2023-10-26', INTERVAL 7 DAY); retornará a data 2023-11-02.

3. DATE\_SUB(data, INTERVAL valor unidade):

Subtrai um intervalo específico de uma data.

data: A data base.

INTERVAL: Palavra-chave obrigatória.

valor: O valor numérico do intervalo.

unidade: A unidade de tempo (ex: DAY, MONTH, YEAR, HOUR, MINUTE, SECOND).

Exemplo: SELECT DATE\_SUB('2023-10-26', INTERVAL 5 DAY); retornará a data 2023-

10-21.

\*/

# Calcular tempo de entrega em dia

SET @varDataInicio = '2021-04-10';

SET @varDataFim = '2021-07-15';

SELECT DATEDIFF(@varDataFim, @varDataInicio);

SET @varDataInicio = '2021-04-10';

SELECT DATE\_ADD(@varDataInicio, INTERVAL 10 DAY); -- Acrescenta 10 dias

SET @varDataInicio = '2021-04-10';

SELECT DATE\_ADD(@varDataInicio, INTERVAL 10 MONTH); -- Acrescenta 10 meses

SET @varDataInicio = '2021-04-10';

SELECT DATE\_ADD(@varDataInicio, INTERVAL 10 YEAR); -- Acrescenta 10 anos

SET @varDataInicio = '2021-04-10';

SELECT DATE\_SUB(@varDataInicio, INTERVAL 10 DAY); -- Diminui 10 dias

-- ++++++

# 40. [MySQL] Joins

# Em MySQL, as funções JOIN são usadas para combinar linhas de duas ou mais tabelas com base em uma coluna relacionada entre elas.

# 1. Chave Primária e Chave Estrangeira

-- A chave primária identifica exclusivamente cada registro em uma tabela.

-- A chave estrangeira estabelece um relacionamento entre tabelas, referenciando a chave primária de outra tabela.

/\*

Chave Primária (Primary Key):

\* É uma coluna ou conjunto de colunas que identifica de forma única cada registro em uma tabela.

\* Não pode conter valores nulos e garante que não haja duplicação de valores na coluna(s) chave.

\* Utilizada para garantir a integridade da entidade.

Exemplo: Em uma tabela de clientes, o ID do cliente pode ser a chave primária.

Chave Estrangeira (Foreign Key):

\* É uma coluna ou conjunto de colunas em uma tabela que faz referência à chave primária de outra tabela.

\* Estabelece um relacionamento entre duas tabelas.

\* Pode conter valores nulos ou não nulos, dependendo da necessidade.

\* Garante a integridade referencial, ou seja, evita que dados inválidos sejam inseridos na tabela com a chave estrangeira.

Exemplo: Em uma tabela de pedidos, o ID do cliente pode ser uma chave estrangeira, referenciando a chave primária da tabela de clientes.

Relação entre Chave Primária e Chave Estrangeira:

A chave estrangeira cria um vínculo entre duas tabelas.

O relacionamento permite que você acesse dados relacionados em diferentes tabelas.

Por exemplo, você pode consultar todos os pedidos de um determinado cliente, utilizando a chave estrangeira na tabela de pedidos que referencia a chave primária da tabela de clientes.

\*/

```

# Exemplo Prático:
-- Criação da tabela de clientes
CREATE TABLE clientes (
    id INT PRIMARY KEY,
    nome VARCHAR(255),
    email VARCHAR(255)
);

-- Criação da tabela de pedidos
CREATE TABLE pedidos (
    id INT PRIMARY KEY,
    cliente_id INT,
    data_pedido DATE,
    FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);
/*

```

Neste exemplo:

clientes.id é a chave primária da tabela clientes.

pedidos.cliente\_id é a chave estrangeira, referenciando a chave primária clientes.id.

Importância:

Garantem a integridade dos dados, evitando inconsistências.

Permite relacionar informações de diferentes tabelas, facilitando consultas e análises.

Otimizam o desempenho do banco de dados ao permitir a criação de índices otimizados para chaves primárias e estrangeiras.

## # 2. Tabela Fato e Tabela Dimensão

Em um sistema de banco de dados MySQL, tabelas fato e tabelas dimensão são usadas para modelagem dimensional, um conceito fundamental para análise de dados. As tabelas fato contêm dados numéricos sobre eventos ou transações, enquanto as tabelas dimensão contêm informações descritivas sobre esses eventos.

Tabelas Fato:

O que são: Armazenam medidas quantitativas de um negócio, como vendas, quantidades, custos, etc.

Características:

- \* Geralmente possuem um grande número de linhas.

- \* Contêm chaves estrangeiras que se referem às tabelas dimensão.

- \* Normalmente contêm dados numéricos que podem ser agregados.

Exemplos: Vendas, cliques, visitas, etc.

Tabelas Dimensão:

O que são: Descrevem o contexto dos dados nas tabelas fato, fornecendo informações adicionais sobre os eventos.

Características:

- \* Geralmente contêm um número menor de linhas em comparação com as tabelas fato.

- \* Contêm atributos textuais que descrevem as dimensões de negócios (quem, o quê, onde, quando, como, por quê).

- \* Cada linha possui uma chave primária única usada para relacionar-se com as tabelas fato.

Exemplos: Produtos, clientes, datas, locais, etc.

Relação entre Tabelas Fato e Dimensão:

- \* As tabelas fato e dimensão são relacionadas através de chaves estrangeiras na tabela fato que se referem às chaves primárias nas tabelas dimensão.

- \* Essa relação permite que os dados sejam analisados sob diferentes perspectivas, combinando informações factuais com informações contextuais.

- \* A estrutura clássica de um modelo dimensional é chamada de esquema em estrela, onde a tabela fato fica no centro, e as tabelas dimensão se conectam a ela.

Exemplo:

- \* Em um sistema de vendas, uma tabela fato poderia conter informações sobre cada venda (ID da venda, ID do produto, ID do cliente, data da venda, quantidade vendida, valor total da venda). As tabelas dimensão poderiam ser:

- \* Tabela Dimensão Produto: Contém informações sobre cada produto (ID do produto, nome do produto, categoria do produto, preço do produto).
- \* Tabela Dimensão Cliente: Contém informações sobre cada cliente (ID do cliente, nome do cliente, endereço, etc.).
- \* Tabela Dimensão Data: Contém informações sobre cada data (ID da data, dia, mês, ano, etc.).

### # 3. JOIN - Contextualização

Em MySQL, a função JOIN permite combinar colunas de duas ou mais tabelas em uma única consulta, baseando-se em colunas relacionadas entre elas. Essa combinação é essencial para recuperar dados que estão espalhados em diferentes tabelas, permitindo análises mais abrangentes e evitando a duplicação de informações.

Tipos de JOINS:

INNER JOIN:

- \* Retorna apenas as linhas que possuem correspondência em ambas as tabelas.

LEFT JOIN:

- \* Retorna todas as linhas da tabela à esquerda (primeira tabela especificada) e as linhas correspondentes da tabela à direita, caso existam. Se não houver correspondência, preenche com NULL.

RIGHT JOIN:

- \* Retorna todas as linhas da tabela à direita (segunda tabela especificada) e as linhas correspondentes da tabela à esquerda, caso existam. Se não houver correspondência, preenche com NULL.

FULL OUTER JOIN:

- \* Retorna todas as linhas de ambas as tabelas, preenchendo com NULL onde não houver correspondência. Embora o MySQL não suporte explicitamente o FULL OUTER JOIN, pode-se obter o mesmo resultado combinando LEFT JOIN e RIGHT JOIN com UNION ALL.

CROSS JOIN:

- \* Cria um produto cartesiano, ou seja, combina cada linha de uma tabela com cada linha da outra tabela.

\*/

-- Sintaxe básica:

SELECT

```
    Tabela_A.coluna1,  
    Tabela_A.coluna2,  
    Tabela_B.coluna1,  
    Tabela_B.coluna1
```

FROM Tabela\_A

INNER JOIN tipo\_de\_join Tabela\_B

```
    ON Tabela_A.coluna_relacionada = Tabela_B.coluna_relacionada;
```

-- Exemplo:

-- Suponha que você tenha duas tabelas: clientes e pedidos. A tabela clientes contém informações sobre os clientes, e a tabela pedidos contém informações sobre os pedidos feitos por esses clientes. Para exibir o nome do cliente e o número do pedido, você pode usar um INNER JOIN:

```
SELECT clientes.nome, pedidos.numero_pedido
```

```
FROM clientes
```

```
INNER JOIN pedidos ON clientes.id = pedidos.cliente_id;
```

-- Este exemplo mostra que o INNER JOIN combina as linhas onde a coluna id da tabela clientes é igual à coluna cliente\_id da tabela pedidos. Isso significa que apenas os clientes que fizeram pedidos serão exibidos, junto com seus respectivos números de pedido.

# Relacionar as tabelas produtos e pedidos do banca base:

```
SELECT * FROM produtos;
```

```
SELECT * FROM pedidos;
```

```
SELECT
```

```
    pedidos.ID_Pedido,  
    pedidos.Data_Venda,  
    pedidos.ID_Produto,  
    pedidos.Qtd_Vendida,  
    pedidos.Receita_Venda,  
    produtos.Nome_Produto,
```

```

        produtos.Marca_Produto
FROM pedidos
INNER JOIN produtos
    ON pedidos.ID_Produto = produtos.ID_Produto;

```

**# Relacionar as tabelas pedidos e clientes do banco base:**

```

SELECT * FROM clientes;
SELECT * FROM pedidos;

```

```

SELECT
    pedidos.ID_Pedido,
    pedidos.Data_Venda,
    pedidos.ID_Cliente,
    pedidos.Qtd_Vendida,
    pedidos.Receita_Venda,
    clientes.Nome,
    clientes.Email,
    clientes.Telefone
FROM pedidos
INNER JOIN clientes
    ON pedidos.ID_Cliente = clientes.ID_Cliente;

```

```

SELECT
    pedidos.ID_Pedido AS 'ID Pedido',
    pedidos.Data_Venda AS 'Data da Venda',
    pedidos.ID_Cliente AS 'ID Cliente',
    pedidos.Qtd_Vendida AS 'QTD. Vendida',
    pedidos.Receita_Venda AS 'Receita das Vendas',
    clientes.Nome AS 'Nome do Cliente',
    clientes.Email AS 'E-mail',
    clientes.Telefone AS 'Telefone'
FROM pedidos
INNER JOIN clientes
    ON pedidos.ID_Cliente = clientes.ID_Cliente;

```

**# 6. Será que sou obrigado a colocar o nome das tabelas antes das colunas**

```

SELECT
    ID_Pedido AS 'ID Pedido',
    Data_Venda AS 'Data da Venda',
    pedidos.ID_Cliente AS 'ID Cliente', -- Somente nos campos comuns nas duas tabelas
    Qtd_Vendida AS 'QTD. Vendida',
    Receita_Venda AS 'Receita das Vendas',
    Nome AS 'Nome do Cliente',
    Email AS 'E-mail',
    Telefone AS 'Telefone'
FROM pedidos
INNER JOIN clientes
    ON pedidos.ID_Cliente = clientes.ID_Cliente;

```

**# Renomeando Tabelas**

```

SELECT
    p.ID_Pedido AS 'ID Pedido',
    p.Data_Venda AS 'Data da Venda',
    p.ID_Cliente AS 'ID Cliente', -- Somente nos campos comuns nas duas tabelas
    p.Qtd_Vendida AS 'QTD. Vendida',
    p.Receita_Venda AS 'Receita das Vendas',
    c.Nome AS 'Nome do Cliente',
    c.Email AS 'E-mail',
    c.Telefone AS 'Telefone'
FROM pedidos AS p
INNER JOIN clientes AS c
    ON p.ID_Cliente = c.ID_Cliente;

```

**# 8. INNER JOIN + WHERE**

```

SELECT

```

```

    p.ID_Pedido AS 'ID Pedido',
    p.Data_Venda AS 'Data da Venda',
    p.ID_Cliente AS 'ID Cliente', -- Somente nos campos comuns nas duas tabelas
    p.Qtd_Vendida AS 'QTD. Vendida',
    p.Receita_Venda AS 'Receita das Vendas',
    c.Nome AS 'Nome do Cliente',
    c.Sexo AS 'Sexo',
    c.Email AS 'E-mail',
    c.Telefone AS 'Telefone'
FROM pedidos AS p
INNER JOIN clientes AS c
    ON p.ID_Cliente = c.ID_Cliente
WHERE c.Sexo = 'M';

```

#### # 9.1. INNER JOIN + GROUP BY

```

SELECT
    Nome_Produto,
    SUM(Receita_Venda) AS 'Receita Total'
FROM pedidos
INNER JOIN produtos
    ON pedidos.ID_Produto = produtos.ID_Produto
GROUP BY Nome_Produto;

```

#### # 9.2. INNER JOIN + GROUP BY

```

SELECT
    Loja,
    SUM(Receita_Venda) AS 'Receita Total',
    SUM(Custo_Venda) AS 'Custo Total'
FROM pedidos
INNER JOIN lojas
    ON pedidos.ID_Loja = lojas.ID_Loja
GROUP BY Loja;

```

/\*

Em MySQL, as funções condicionais IF, IFNULL, ISNULL e NULLIF, além da estrutura CASE, são usadas para controlar o fluxo de execução e lidar com valores NULL em consultas.

Funções Condicionais:

IF(condicao, valor\_verdadeiro, valor\_falso):

\* Avalia uma condição e retorna um valor se a condição for verdadeira e outro valor se for falsa.

IFNULL(expressao, valor\_alternativo):

\* Verifica se uma expressão é NULL. Se for, retorna o valor alternativo; caso contrário, retorna o valor da expressão original.

ISNULL(expressao):

\* Verifica se uma expressão é NULL. Retorna 1 se for NULL e 0 caso contrário. Similar ao IFNULL mas retorna um valor booleano.

NULLIF(expressao1, expressao2):

\* Retorna NULL se as duas expressões forem iguais; caso contrário, retorna a primeira expressão.

Estrutura CASE:

\* A estrutura CASE permite avaliações mais complexas com múltiplas condições, Sintaxe.

```

CASE
    WHEN condicao1 THEN resultado1
    WHEN condicao2 THEN resultado2
    ...
    ELSE resultado_padrao
END

```

\* A estrutura CASE avalia as condições em ordem e retorna o resultado correspondente à primeira condição verdadeira. Se nenhuma condição for verdadeira, retorna o resultado do ELSE. Se o ELSE não for especificado e nenhuma condição for verdadeira, retorna NULL.

\*/

# Exemplos Práticos:

-- IF:

```

SELECT IF(10 > 12, 'VERDADEIRO', 'FALSO') AS 'TESTES';

```

```

-- ++++++
SET @varNotaDesempenho = 6;
SELECT
    IF(@varNotaDesempenho >= 7, 'Recebe Bonus de 10%',
    IF(@varNotaDesempenho >= 5, 'Recebe Bonus de 5%',
    'Não recebe Bônus'));
-- ++++++
SET @varIdade = 2025;
SELECT Nome, Data_Nascimento, IF((@varIdade - YEAR(Data_Nascimento)) >= 28,
'Maior de idade', 'Menor de idade') AS faixa_etaria FROM clientes;
-- IFNULL:
SELECT IFNULL(100, 'Valor Alternativo');
SELECT Nome, IFNULL(Telefone, 'Telefone não informado') AS telefone FROM
clientes;
-- ISNULL:
SELECT *, ISNULL(Telefone) AS Telefone_Nulo FROM clientes;
SELECT *, IF(ISNULL(Telefone), 'Verificar Fone', 'Fone OK') FROM clientes;
-- NULLIF:
SELECT NULLIF(20, 30) FROM clientes;
-- ++++++
SET @var01 = 500;
SET @var01 = 100;
SELECT NULLIF(@var01, @var02);
-- CASE:
SELECT
    Nome_Produto,
    Preco_Unit,
    CASE
        WHEN Preco_Unit <= 400 THEN 'Baixo'
        WHEN Preco_Unit <= 700 THEN 'Médio'
        WHEN Preco_Unit <= 1000 THEN 'Alto'
        ELSE 'Pesado'
    END AS faixa_valor
FROM produtos;
-- ++++++
SELECT *,
    CASE
        WHEN Sexo = 'M' THEN 'Masculino'
        ELSE 'Feminino'
    END AS 'Sexo 2'
FROM clientes;
-- ++++++
SELECT *,
    CASE
        WHEN Receita_Venda >= 3000 THEN 'Faturamento Alto'
        WHEN Receita_Venda >= 1000 AND Receita_Venda < 3000 THEN 'Faturamento
Médio'
        ELSE 'Faturamento Baixo'
    END AS 'Status'
FROM pedidos;
-- ++++++
# 4. Função CASE + AND
SELECT *,
    CASE
        WHEN Sexo = 'F' AND Qtd_Filhos > 0 THEN 'Oferta dia das Mães'
        WHEN Sexo = 'M' AND Qtd_Filhos > 0 THEN 'Oferta dia das Pais'
        ELSE 'Sem oferta'
    END AS 'Situação'
FROM clientes;
-- ++++++
SELECT
    *,
    CASE
        WHEN Marca_Produto = 'DELL' OR Marca_Produto = 'SAMSUNG' THEN
'Recebe Desconto de 15%'

```

```

        ELSE 'Não recebe desconto'
      END AS 'Desconto'
    FROM produtos;
-- ++++++
SELECT
    *,
    CASE
      WHEN Marca_Produto IN('DELL', 'SAMSUNG') THEN (1 - 0.15) *
Custo_Unit
      ELSE Custo_Unit
    END AS 'Desconto'
  FROM produtos;
-- ++++++
/*

```

#### Observações:

- \* IF, IFNULL e NULLIF são funções, enquanto CASE é uma estrutura.
- \* IFNULL e NULLIF são funções específicas do MySQL. COALESCE é uma função padrão SQL que funciona de maneira semelhante ao IFNULL, mas pode receber mais de dois argumentos.
- \* A estrutura CASE é uma construção mais poderosa para lidar com múltiplas condições em comparação com IF.
- \* É importante entender a diferença entre NULL e 0 ou "" (string vazia). NULL representa a ausência de valor, enquanto 0 e "" são valores válidos.

## # 42. [MySQL] Views

### # 1. Introdução

Sim, no MySQL, views são objetos que permitem criar uma representação virtual de dados armazenados em uma ou mais tabelas. Elas não armazenam dados fisicamente, mas sim uma consulta SQL que é executada quando a view é acessada. Isso oferece diversas vantagens, como simplificar o acesso a dados complexos, implementar segurança e prover isolamento para as aplicações.

#### O que são Views?

-- Views são tabelas virtuais que exibem dados de outras tabelas ou até mesmo de outras views. Elas são criadas usando a instrução CREATE VIEW seguida de uma consulta SQL SELECT. A consulta define quais colunas e linhas serão exibidas na view.

#### Benefícios das Views:

##### Simplificação:

\* Permitem criar uma visualização mais simples de dados complexos, facilitando o acesso e a compreensão.

##### Segurança:

\* Podem restringir o acesso a certos dados, protegendo informações sensíveis.

##### Isolamento:

\* Protegem as aplicações da estrutura física do banco de dados, permitindo alterações nas tabelas sem afetar as aplicações que utilizam as views.

##### Reutilização:

\* Consultas complexas podem ser encapsuladas em views, permitindo que sejam reutilizadas em diferentes partes do sistema.

##### Atualizabilidade:

\* Em alguns casos, views podem ser atualizáveis, permitindo que operações de inserção, atualização e exclusão sejam realizadas através delas.

Este exemplo cria uma view chamada clientes\_masculinos que exibe apenas os clientes com status "ativo" da tabela clientes.

#### Desvantagens das Views:

Complexidade: Em alguns casos, views podem adicionar complexidade ao sistema, especialmente se forem muito complexas ou aninhadas.

Desempenho: Consultas que envolvem muitas views ou views muito complexas podem ter impacto no desempenho.

Manutenção: Alterações nas tabelas base podem exigir ajustes nas views.

Em resumo, views são ferramentas poderosas no MySQL para simplificar o acesso aos dados, implementar segurança e proteger as aplicações da estrutura do banco de dados. No entanto, é importante avaliar cuidadosamente a necessidade de usar views e considerar seus potenciais impactos no desempenho e na manutenção.

## # 2. CREATE VIEW

Exemplo de Criação de View: \*/

```
CREATE VIEW vwClientes_masculinos AS
SELECT ID_Cliente, Nome, Data_Nascimento, Sexo, Email, Telefone
FROM clientes
WHERE Sexo = 'M'; -- Atualiza Schema e vê em Views
```

```
SELECT * FROM vwClientes_masculinos;
```

## # 3. ALTER VIEW e DROP VIEW

```
ALTER VIEW vwClientes_masculinos AS
SELECT ID_Cliente, Nome, Data_Nascimento, Sexo, Email, Telefone, Escolaridade
FROM clientes
WHERE Escolaridade = 'Parcial'; -- Atualiza Schema e vê em Views
```

```
SELECT * FROM vwClientes_masculinos;
```

## # Excluir uma VIEW - 3. DROP VIEW

```
DROP VIEW vwclientes_masculinos;
-- ++++++
```

## # 4. Exemplo 1 - Criando uma View com o comando WHERE

```
SELECT * FROM pedidos;

CREATE VIEW vwReceitaAcima4000 AS
SELECT * FROM pedidos
WHERE Receita_Venda >= 4000; -- Atualizar schema
```

```
SELECT * FROM vwReceitaAcima4000;
```

## # Excluir uma VIEW - 3. DROP VIEW

```
DROP VIEW vwReceitaAcima4000;

-- ++++++
SELECT * FROM pedidos;
SELECT * FROM vwReceitaAcima4000;
```

```
ALTER VIEW vwReceitaAcima4000 AS
SELECT * FROM pedidos
WHERE Receita_Venda >= 4000 AND ID_Loja = 2; -- Atualizar schema
```

```
SELECT * FROM vwReceitaAcima4000;
```

## # Excluir uma VIEW - 3. DROP VIEW

```
DROP VIEW vwReceitaAcima4000;

-- ++++++
SELECT * FROM pedidos;
SELECT * FROM vwReceitaAcima4000;
```

```
ALTER VIEW vwReceitaAcima4000 AS
SELECT * FROM pedidos
WHERE Receita_Venda >= 4000 AND ID_Loja IN(2, 4); -- Atualizar schema
```

```
SELECT * FROM vwReceitaAcima4000;
```

## # Excluir uma VIEW - 3. DROP VIEW

```
DROP VIEW vwReceitaAcima4000;

-- ++++++
SELECT * FROM produtos;
```

```
CREATE VIEW vwProdutosAtualizados AS
SELECT * FROM produtos
WHERE Marca_Produto IN('DELL', 'SAMSUNG', 'SONY'); -- Atualizar schema
```

```

SELECT * FROM vwProdutosAtualizados;

# Excluir uma VIEW - 3. DROP VIEW
DROP VIEW vwProdutosAtualizados;

-- ++++++
SELECT * FROM pedidos;

CREATE VIEW vwReceitaCustoTotal AS
SELECT
    ID_Produto,
    SUM(Receita_Venda) AS 'Total Receita' ,
    SUM(Custo_Venda) AS 'Total Custo'
FROM pedidos
GROUP BY ID_Produto; -- Atualizar schema

SELECT * FROM vwReceitaCustoTotal;

-- ++++++
SELECT * FROM vwReceitaCustoTotal;

ALTER VIEW vwReceitaCustoTotal AS
SELECT
    ID_Produto,
    ID_Loja,
    SUM(Receita_Venda) AS 'Total Receita' ,
    SUM(Custo_Venda) AS 'Total Custo'
FROM pedidos
WHERE ID_Loja = 2
GROUP BY ID_Produto; -- Atualizar schema

SELECT * FROM vwReceitaCustoTotal;

-- ++++++
SELECT * FROM vwReceitaCustoTotal;

ALTER VIEW vwReceitaCustoTotal AS
SELECT
    ID_Produto,
    ID_Loja,
    SUM(Receita_Venda) AS 'Total Receita' ,
    SUM(Custo_Venda) AS 'Total Custo'
FROM pedidos
WHERE ID_Loja = 2
GROUP BY ID_Produto
HAVING SUM(Receita_Venda) >= 1000000; -- Alterado
-- Atualizar schema
SELECT * FROM vwReceitaCustoTotal;

# Excluir uma VIEW - 3. DROP VIEW
DROP VIEW vwReceitaCustoTotal;

# 6. Exemplo 3 - Criando uma View com os comandos INNER JOIN e GROUP BY
SELECT * FROM pedidos;
SELECT * FROM produtos;

CREATE VIEW vwPedidosCompleta AS
SELECT
    pe.*,
    pr.Nome_Produto,
    pr.Marca_Produto,
    pr.Num_Serie
FROM pedidos AS pe
INNER JOIN produtos AS pr

```

```

ON pe.ID_Produto = pr.ID_Produto; -- Atualizar schema

SELECT * FROM vwPedidosCompleta;
# Excluir uma VIEW - 3. DROP VIEW
DROP VIEW vwPedidosCompleta;

# 6. Exemplo 3 - Criando uma View com os comandos INNER JOIN e GROUP BY
SELECT * FROM pedidos;
SELECT * FROM produtos;

CREATE VIEW vwResultadoFianl AS
SELECT
    pr.Nome_Produto,
    SUM(Receita_Venda) AS 'Receita_Total',
    SUM(Custo_Venda) AS 'Custo_Total'
FROM pedidos AS pe
INNER JOIN produtos AS pr
    ON pe.ID_Produto = pr.ID_Produto
GROUP BY Nome_Produto; -- Atualizar schema

SELECT * FROM vwResultadoFianl;
# Excluir uma VIEW - 3. DROP VIEW
DROP VIEW vwResultadoFianl;

# 43. [MySQL] CRUD
# 1. O que significa CRUD
/*
Em MySQL, CRUD é um acrônimo para as quatro operações básicas de gerenciamento de
dados: Create (Criar), Read (Ler), Update (Atualizar) e Delete (Deletar). Essas
operações são fundamentais para interagir com um banco de dados e manipular as
informações armazenadas nele.
Entendendo cada operação:
Create (Criar):
    * Permite a inserção de novos dados no banco de dados. Em SQL, isso é feito com
o comando INSERT.
Read (Ler):
    * Permite a consulta e recuperação de dados existentes no banco de dados. Em
SQL, isso é feito com o comando SELECT.
Update (Atualizar):
    * Permite a modificação de dados existentes no banco de dados. Em SQL, isso é
feito com o comando UPDATE.
Delete (Deletar):
    * Permite a remoção de dados do banco de dados. Em SQL, isso é feito com o
comando DELETE.
Importância do CRUD em MySQL:
    * O CRUD é essencial para o desenvolvimento de qualquer aplicação que utilize um
banco de dados, pois ele fornece as operações básicas para manipular as informações,
como criar novos registros, visualizar dados existentes, atualizar informações e
remover dados desnecessários.
Exemplos em SQL:
*/
-- C = Create (CREATE DATABASE)
-- O comando CREATE DATABASE é seguido pelo nome desejado para o novo banco de
dados. Por exemplo, CREATE DATABASE nome_do_banco; irá criar um banco de dados chamado
"nome_do_banco".
-- Exemplo:
CREATE DATABASE db_Exemplo;
-- Este comando criará um banco de dados chamado "minha_loja".
-- Recursos adicionais:
-- É recomendado utilizar a cláusula IF NOT EXISTS para evitar erros caso o
banco de dados já exista.
CREATE DATABASE IF NOT EXISTS db_Exemplo; -- Verifica se o BD já existe.
-- Para utilizar o comando CREATE DATABASE, o usuário precisa ter os privilégios
necessários.
SHOW DATABASES; -- Verifica/mostra todos os Banco de Dados Existentes.

```

```

-- O comando SHOW DATABASES; pode ser usado para verificar todos os bancos de
dados existentes no servidor MySQL.
USE db_Exemplo; -- Define o BD a ser usado.
-- Após a criação do banco de dados, você pode usar o comando USE nome_do_banco;
para começar a trabalhar com ele.
SELECT DATABASE(); -- Verifica o BD selecionado.
DROP DATABASE db_Exemplo; -- Exclui/Apaga BD.
DROP DATABASE IF EXISTS db_Exemplo; -- Exclui/Apaga BD SE EXISTIR.

```

```

-- C = Create (INSERT)
INSERT INTO tabela (coluna1, coluna2) VALUES ('valor1', 'valor2');

```

```

-- R = Read (SELECT)
SELECT * FROM tabela WHERE condição;

```

```

-- U = Update (UPDATE)
UPDATE tabela SET coluna1 = 'novo_valor' WHERE condição;

```

```

-- D = Delete (DELETE)
DELETE FROM tabela WHERE condição;
-- Esses comandos são exemplos de como as operações CRUD são realizadas em SQL,
utilizando o MySQL como banco de dados.

```

```

-- ++++++
/*

```

Em MySQL, CREATE TABLE é o comando usado para criar novas tabelas em um banco de dados, definindo sua estrutura com nomes de colunas, tipos de dados e restrições. Os tipos de dados em MySQL determinam o tipo de informação que cada coluna pode armazenar, como números inteiros, texto, datas, etc.

CREATE TABLE

O comando CREATE TABLE é usado para criar uma nova tabela no banco de dados. A sintaxe básica é:

```

CREATE TABLE nome_tabela (
    nome_coluna1 tipo_dado1 restrições1,
    nome_coluna2 tipo_dado2 restrições2,
    ...
);

```

nome\_tabela: O nome que você dará à tabela.

nome\_coluna: O nome de cada coluna na tabela.

tipo\_dado: O tipo de dados que a coluna irá armazenar (ex: INT, VARCHAR, DATE).

restrições: Condições adicionais que a coluna deve seguir (ex: PRIMARY KEY, NOT NULL, UNIQUE).

### Tipos de Dados em MySQL

MySQL oferece uma variedade de tipos de dados para armazenar diferentes tipos de informação. Aqui estão alguns dos tipos mais comuns:

Tipos Numéricos:

INT ou INTEGER: Números inteiros.

\* TINYINT, SMALLINT, MEDIUMINT, BIGINT: Variações de tamanho para números inteiros.

DECIMAL ou NUMERIC: Números decimais com precisão definida.

FLOAT: Números de ponto flutuante de precisão simples.

DOUBLE ou REAL: Números de ponto flutuante de precisão dupla.

Tipos de Texto:

CHAR: Uma string de comprimento fixo.

VARCHAR: Uma string de comprimento variável.

TEXT: Uma string longa.

BLOB: Um tipo de dados binário para armazenar grandes quantidades de dados binários.

Tipos de Data e Hora:

DATE: Armazena a data (AAAA-MM-DD).

TIME: Armazena a hora (HH:MM:SS).

DATETIME: Armazena a data e a hora.

TIMESTAMP: Armazena a data e a hora, atualizado automaticamente quando a linha é

alterada.

YEAR: Armazena o ano.

Tipos Especiais:

BOOLEAN ou BOOL: Armazena valores booleanos (VERDADEIRO ou FALSO).

ENUM: Uma lista de valores permitidos.

SET: Permite que uma coluna armazene vários valores de uma lista pré-definida.

JSON: Armazena dados no formato JSON.

Exemplo:

```
CREATE TABLE clientes (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(255) NOT NULL,  
    email VARCHAR(255) UNIQUE,  
    data_nascimento DATE,  
    ativo BOOLEAN DEFAULT TRUE  
);
```

Neste exemplo, a tabela clientes tem colunas para ID (chave primária), nome (texto), e-mail (texto único), data de nascimento e um valor booleano para indicar se o cliente está ativo (com valor padrão TRUE).

\*/

```
SELECT DATABASE(); -- Verifica o BD selecionado.
```

```
# CRIAR BD E TABELAS EXEMPLO
```

```
-- 1º - Criar BD db_exemplo se não existir
```

```
CREATE DATABASE IF NOT EXISTS db_Exemplo;
```

```
-- 2º - Usar o BD db_Exemplo
```

```
USE db_Exemplo; -- Define o BD a ser usado.
```

```
SELECT DATABASE(); -- Verifica o BD selecionado.
```

```
-- 3º - Criar a tabela dAlunos. Verificar se existe.
```

```
CREATE TABLE IF NOT EXISTS tb_Alunos (  
    ID_Aluno INT,  
    Nome_Aluno VARCHAR(100),  
    Email VARCHAR(100),  
    Data_Nascimento DATE  
);
```

```
);
```

```
-- 4º - Criar a tabela dCursos. Verificar se existe.
```

```
CREATE TABLE IF NOT EXISTS tb_Cursos (  
    ID_Curso INT,  
    Nome_Curso VARCHAR(100),  
    Preço_Curso DECIMAL(10, 2) -- 99999999.99  
);
```

```
);
```

```
-- 5º - Criar a tabela dMatriculas. Verificar se existe.
```

```
CREATE TABLE IF NOT EXISTS tb_Matriculas (  
    ID_Matricula INT,  
    ID_Aluno INT,  
    ID_Curso INT,  
    Data_Cadastro DATE  
);
```

```
);
```

```
-- Mostrar Tabelas
```

```
SHOW TABLES;
```

```
-- Se for preciso deletar uma tabela
```

```
DROP TABLE tb_Alunos;
```

```
SHOW TABLES;
```

```
DROP TABLE IF EXISTS tb_Matriculas;
```

```
SHOW TABLES;
```

```
DROP TABLE IF EXISTS tb_Cursos;
```

```
SHOW TABLES;
```

```
-- Deletar BD
```

```
SELECT DATABASE(); -- Verifica o BD selecionado.
```

```
DROP DATABASE IF EXISTS db_Exemplo; -- Exclui/Apaga BD SE EXISTIR.
```

```
SELECT DATABASE(); -- Verifica o BD selecionado.
```

```
-- Consulta Tabelas
```

```
SELECT * FROM tb_Alunos;
```

```
SELECT * FROM tb_cursos;
```

```
SELECT * FROM tb_matriculas;
```

```
/*
```

4. Constraints – O que são e como usar

Em MySQL, constraints (restrições) são regras que você define para as colunas de uma tabela para garantir a integridade e a validade dos dados. Elas atuam como mecanismos de controle, impedindo a inserção de dados inválidos e garantindo que os dados armazenados sigam padrões específicos.

Em outras palavras, constraints são como "regras de trânsito" para o seu banco de dados, garantindo que apenas informações corretas e consistentes sejam inseridas.

Principais tipos de constraints em MySQL:

NOT NULL:

- \* Garante que uma coluna não pode conter valores nulos (vazios).

UNIQUE:

- \* Impede que valores duplicados sejam inseridos em uma coluna, garantindo que cada valor seja único.

PRIMARY KEY:

- \* Combina as características de NOT NULL e UNIQUE, identificando exclusivamente cada registro em uma tabela. Uma tabela só pode ter uma chave primária.

FOREIGN KEY:

- \* Estabelece um relacionamento entre duas tabelas, referenciando a chave primária de outra tabela. Isso garante a integridade referencial, impedindo a criação de referências inválidas.

CHECK:

- \* Permite definir uma condição que deve ser verdadeira para todos os valores inseridos em uma coluna.

DEFAULT:

- \* Define um valor padrão para uma coluna caso nenhum valor seja especificado durante a inserção.

ENUM:

- \* Restringe os valores possíveis em uma coluna a uma lista predefinida de valores.

Por que usar constraints?

Integridade dos dados:

- \* Garante que os dados armazenados sejam precisos e consistentes.

Validação de dados:

- \* Impede a inserção de dados inválidos, evitando erros e inconsistências.

Facilita a manutenção:

- \* Facilita a identificação e correção de problemas relacionados à qualidade dos dados.

Melhora a performance:

- \* Em alguns casos, constraints podem otimizar o desempenho do banco de dados.

Ao usar constraints, você garante que seu banco de dados seja mais robusto, confiável e fácil de gerenciar.

```
*/
```

```
SELECT DATABASE(); -- Verifica o BD selecionado.
```

```
# CRIAR BD E TABELAS EXEMPLO
```

```
-- 1º - Criar BD db_exemplo se não existir
```

```
CREATE DATABASE IF NOT EXISTS db_Exemplo;
```

```
-- 2º - Usar o BD db_Exemplo
```

```
USE db_Exemplo; -- Define o BD a ser usado.
```

```
SELECT DATABASE(); -- Verifica o BD selecionado.
```

```
-- Mostrar Tabelas
```

```
SHOW TABLES;
```

```
-- 3º - Criar a tabela dAlunos. Verificar se existe.
```

```
CREATE TABLE IF NOT EXISTS tb_Alunos (
```

```
    ID_Aluno INT,
```

```
    Nome_Aluno VARCHAR(100) NOT NULL,
```

```
    Email VARCHAR(100) NOT NULL,
```

```
    Data_Nascimento DATE,
```

```
    PRIMARY KEY(ID_Aluno)
```

```
);
```

```
-- Atualizar SCHEMAS
```

```
-- 4º - Criar a tabela dCursos. Verificar se existe.
```

```
CREATE TABLE IF NOT EXISTS tb_Cursos (
```

```

        ID_Curso INT,
        Nome_Curso VARCHAR(100) NOT NULL,
        Preco_Curso DECIMAL(10, 2) NOT NULL, -- 99999999.99
        PRIMARY KEY(ID_Curso)
    );
-- 5° - Criar a tabela dMatriculas. Verificar se existe.
CREATE TABLE IF NOT EXISTS tb_Matriculas (
    ID_Matricula INT,
    ID_Aluno INT NOT NULL,
    ID_Curso INT NOT NULL,
    Data_Cadastro DATE NOT NULL,
    PRIMARY KEY(ID_Matricula),
    FOREIGN KEY(ID_Aluno) REFERENCES tb_Alunos(ID_Aluno),
    FOREIGN KEY(ID_Curso) REFERENCES tb_Cursos(ID_Curso)
);
-- Mostrar Tabelas
SHOW TABLES;
-- Se for preciso deletar uma tabela
DROP TABLE tb_Alunos;
SHOW TABLES;
DROP TABLE IF EXISTS tb_Matriculas;
SHOW TABLES;
DROP TABLE IF EXISTS tb_Cursos;
SHOW TABLES;
-- Deletar BD
SELECT DATABASE(); -- Verifica o BD selecionado.
DROP DATABASE IF EXISTS db_Exemplo; -- Exclui/Apaga BD SE EXISTIR.
SELECT DATABASE(); -- Verifica o BD selecionado.
-- Consulta Tabelas
SELECT * FROM tb_Alunos;
SELECT * FROM tb_cursos;
SELECT * FROM tb_matriculas;
# 5. Visualizando relacionamentos entre tabelas
# Para ver o modelo Relaciona das Tabelas
-- Clique na Casinha
-- Clique em Models > e escolha a opção CREATE EER Model From Database ==> Next
-- Stored Connection: Local instance MySQL80 ==> Next
-- Senha do BD root: j2cm1928 ==> Next
-- Escolha o BD para montar o Modelo Relacional ==> Next ==> Next
-- Será mostrado o modelo relacional
# 6. INSERT INTO - Inserindo dados em uma tabela no banco de dados
INSERT INTO tb_Alunos(ID_Aluno, Nome_Aluno, Email, Data_Nascimento)
VALUES (1, 'Ana', 'ana@gmail.com', '2004-08-25'),
       (2, 'Bruno', 'bruno@gmail.com', '2001-06-15'),
       (3, 'Carla', 'carla@gmail.com', '2002-05-26'),
       (4, 'Diego', 'diego@gmail.com', '2000-07-05'),
       (5, 'Charmilton', 'charmilton@gmail.com', '1976-03-11'),
       (6, 'Analia', 'analia@gmail.com', '2019-03-08');
SELECT * FROM tb_Alunos;
-- ++++++
INSERT INTO tb_Cursos(ID_Curso, Nome_Curso, Preco_Curso)
VALUES (1, 'Excel', 100.00),
       (2, 'VBA', 200.00),
       (3, 'Power BI', 250.00),
       (4, 'Python', 1150.00);
SELECT * FROM tb_cursos;
-- ++++++
INSERT INTO tb_matriculas(ID_Matricula, ID_Aluno, ID_Curso, Data_Cadastro)
VALUES (1, 5, 4, '2025-01-02'),
       (2, 5, 2, '2025-01-02'),
       (3, 6, 1, '2025-02-21'),
       (4, 6, 4, '2025-02-21');
SELECT * FROM tb_matriculas;
# 7. UPDATE - Atualizar registros de uma tabela
-- Consulta Tabelas

```

```
SELECT * FROM tb_Alunos;
SELECT * FROM tb_cursos;
SELECT * FROM tb_matriculas;
```

```
--
```

```
UPDATE tb_Cursos
    SET Preco_Curso = 333.50
WHERE ID_Curso = 1;
```

```
--
```

```
SELECT * FROM tb_cursos;
```

# 8. DELETE - Deletando registros de uma tabela

```
DELETE FROM tb_Alunos
    WHERE ID_Aluno = 1;
```

```
--
```

```
SELECT * FROM tb_Alunos;
```

# 9. TRUNCATE TABLE x DROP TABLE

```
/*
```

Em MySQL, TRUNCATE TABLE e DROP TABLE são comandos para remover dados de uma tabela, mas com efeitos e propósitos diferentes. TRUNCATE TABLE remove todos os dados de uma tabela, mantendo sua estrutura (esquema), enquanto DROP TABLE remove a tabela inteira, incluindo dados e estrutura.

TRUNCATE TABLE:

- \* Remove todos os registros de uma tabela, mas a tabela em si, suas colunas, índices e outras propriedades permanecem intactas.
- \* É mais rápido que DELETE para remover todos os dados, pois não gera logs detalhados.
- \* Redefine contadores de identidade (autoincremento) para o valor inicial.
- \* Pode ser usado dentro de transações e pode ser revertido, mas com cuidado.
- \* Útil quando se deseja limpar o conteúdo de uma tabela sem remover sua estrutura.

DROP TABLE:

- \* Remove a tabela do banco de dados, incluindo todos os seus dados, colunas, índices, restrições e outros objetos associados.
- \* É uma operação mais destrutiva, pois apaga a tabela permanentemente do banco de dados.
- \* Não pode ser desfeita (em MySQL) após a execução.
- \* Útil quando a tabela não é mais necessária e se deseja liberar o espaço ocupado por ela.

```
*/
```

```
-- TRUNCATE TABLE: Remove todos os registros de uma tabela, mas a tabela em si, suas colunas, índices e outras propriedades permanecem intactas.
```

```
TRUNCATE TABLE tb_Matriculas;
```

```
-- DROP TABLE: Remove a tabela do banco de dados, incluindo todos os seus dados, colunas, índices, restrições e outros objetos associados.
```

```
DROP TABLE tb_Matriculas;
```

```
-- ++++++
```

# 44. [MySQL] Functions e Stored Procedures

# 1. Functions - Introdução

```
/*
```

Em MySQL, funções são criadas utilizando o comando CREATE FUNCTION seguido do nome da função, parâmetros de entrada e um bloco de código que define o que a função faz e o valor de retorno. O bloco de código é delimitado por BEGIN e END e pode conter instruções SQL para manipular dados ou realizar cálculos.

Sintaxe básica:

```
-- ++++++
```

```
DELIMITER $$
```

```
CREATE FUNCTION nome_da_funcao (parametro tipo_dado, ...)
```

```
RETURNS tipo_dado_de_retorno
```

```
BEGIN
```

```
    -- Código da função (instruções SQL)
```

```
    -- Exemplo:
```

```

-- DECLARE variavel tipo_dado;
-- SET variavel = valor;
-- RETURN variavel;
END $$

DELIMITER ;
-- ++++++
Exemplo:
-- ++++++
*/
DELIMITER $$
CREATE FUNCTION dobrar_numero(numero INT)
RETURNS INT DETERMINISTIC
BEGIN
    DECLARE resultado INT;
    SET resultado = numero * 2;
    RETURN resultado;
END $$

```

```

DELIMITER ;
-- Atualiza o SCHEMAS
-- Chamando a função:
SELECT dobrar_numero(20); -- Retorna 40
SELECT dobrar_numero(5); -- Retorna 10
SELECT dobrar_numero(15); -- Retorna 30
-- ++++++
/*

```

Observações:

O delimitador (neste caso, \$\$) é usado para permitir que o DELIMITER seja usado dentro da função sem ser interpretado pelo cliente MySQL.

RETURNS tipo\_dado\_de\_retorno define o tipo de dado que a função irá retornar.

O bloco BEGIN ... END contém o código da função, incluindo declarações de variáveis, manipulação de dados e o comando RETURN para especificar o valor de retorno.

É possível usar funções armazenadas em expressões SQL, como em SELECT, WHERE, ou ORDER BY.

Funções armazenadas podem ser usadas para encapsular lógica de negócio, cálculos complexos ou operações repetitivas, tornando o código mais organizado e reutilizável. Existem diferentes tipos de funções no MySQL, incluindo funções armazenadas (criadas com CREATE FUNCTION), funções carregáveis (carregadas dinamicamente) e funções nativas (incorporadas ao servidor).

\*/

# Criar uma função que retorne a msg 'Olá Mundo!!!!'

```

DELIMITER $$
CREATE FUNCTION fn_BoasVindas(frase VARCHAR(100))
RETURNS VARCHAR(100) DETERMINISTIC
BEGIN
    RETURN CONCAT('Olá ', frase, ', tudo bem?');
END $$

```

```

DELIMITER ;
-- Atualizar SCHEMAS e verificar em Functions.
-- Chamando a função:
SELECT fn_BoasVindas('José'); -- Retorna 'Olá José , tudo bem?'
SELECT fn_BoasVindas('Charles'); -- Retorna 'Olá Charles , tudo bem?'
-- ++++++

```

# 3. Functions – Exemplo 2

```

DELIMITER $$
CREATE FUNCTION fn_Faturamento(preco DECIMAL(10, 2), quantidade INT)
RETURNS DECIMAL(10, 2) DETERMINISTIC
BEGIN
    RETURN preco * quantidade;
END $$

```

```

DELIMITER ;
-- Atualiza o SCHEMAS
-- Chamando a função:
SELECT fn_Faturamento(20.50, 10); -- Retorna 205.00
SELECT fn_Faturamento(5.35, 15); -- Retorna 80.25

```

```

SELECT fn_Faturamento(15.40, 10); -- Retorna 154.00
-- ++++++
-- Usar o BD base
USE base;
# 4. Functions - Exemplo 3
# Retirar caracteres especiais de um texto.
DELIMITER $$

```

```

CREATE FUNCTION remover_acentos(texto VARCHAR(255))
RETURNS VARCHAR(255)
DETERMINISTIC
BEGIN
    DECLARE texto_sem_acentos VARCHAR(255);

    -- Substituição manual dos caracteres acentuados
    SET texto_sem_acentos = texto;

    -- Letras minúsculas
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'á', 'a');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'à', 'a');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ã', 'a');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'â', 'a');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ä', 'a');

    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'é', 'e');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'è', 'e');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ê', 'e');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ë', 'e');

    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'í', 'i');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ì', 'i');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'î', 'i');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ï', 'i');

    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ó', 'o');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ò', 'o');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'õ', 'o');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ô', 'o');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ö', 'o');

    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ú', 'u');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ù', 'u');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'û', 'u');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ü', 'u');

    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ç', 'c');

    -- Letras maiúsculas
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Á', 'A');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'À', 'A');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Ã', 'A');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Â', 'A');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Ä', 'A');

    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'É', 'E');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'È', 'E');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Ê', 'E');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Ë', 'E');

    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Í', 'I');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Ì', 'I');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Î', 'I');
    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Ï', 'I');

    SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'Ó', 'O');

```

```

SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ò', 'o');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'õ', 'o');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ô', 'o');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ö', 'o');

SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ú', 'u');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ù', 'u');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'û', 'u');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ü', 'u');

SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ç', 'c');

-- Remover outros caracteres especiais comuns
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '°', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'ª', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, 'º', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '§', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '@', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '!', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '?', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '#', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '$', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '%', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '&', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '*', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '(', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, ')', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '[', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, ']', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '{', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '}', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '<', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '>', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '~', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '^', '');
SET texto_sem_acentos = REPLACE(texto_sem_acentos, '`', '');

RETURN texto_sem_acentos;
END $$

DELIMITER ;
-- Atualizar SCHEMAS
-- Resultado esperado: 'Ola, Joao Voce esta com cafe Acucar nao'
SELECT remover_acentos('Olá, João! Você está com café? Açúcar não! Andréia.');
```

--

```

SELECT * FROM lojas;
SELECT
    Loja,
    Endereco,
    remover_acentos(Endereco) -- Retira os acentos de endereco
FROM lojas;
```

# 5. Alter function - Alterando uma função criada

```

-- Para alterar uma Função, clique com o botão direito sobre a Função
'remover_acentos'
-- Opção 'Alter Function'
-- Fazer a alteração ==> Aply ==> apply(aplicar ==> aplicar).
```

/\*

# 7. Stored Procedures - Introdução e exemplo

Em MySQL, procedimentos armazenados (Stored Procedures) são conjuntos de instruções SQL pré-compiladas que podem ser executadas com um único comando. Eles permitem encapsular lógica de negócio complexa dentro do banco de dados, melhorando a segurança, desempenho e manutenção do código.

Introdução aos Stored Procedures:

O que são:

- \* Procedimentos armazenados são rotinas armazenadas no banco de dados que podem

receber parâmetros, executar uma sequência de comandos SQL e retornar resultados.

Benefícios:

- \* Reutilização de código: Evitam a repetição de código SQL em diferentes partes da aplicação.

Segurança:

- \* Permitem restringir o acesso a dados e operações, utilizando privilégios específicos para cada procedimento.

Desempenho:

- \* Reduzem a quantidade de dados transferidos entre a aplicação e o banco de dados, pois a lógica é executada dentro do servidor de banco de dados.

Manutenção:

- \* Facilita a atualização da lógica de negócio, pois as alterações são feitas em um único local (o procedimento armazenado).

Como funcionam:

- \* São definidos usando a instrução CREATE PROCEDURE, com um nome, parâmetros (opcional) e um bloco de código BEGIN...END que contém as instruções SQL.

Chamada:

- \* São executados utilizando o comando CALL seguido do nome do procedimento e dos parâmetros, se houver.

Exemplo:

```
-- Cria um procedimento armazenado chamado 'get_employees_by_city'
```

```
*/
```

```
DELIMITER $$
```

```
CREATE PROCEDURE get_employees_by_city(IN city_name VARCHAR(255))
```

```
BEGIN
```

```
    -- Seleciona todos os funcionários que trabalham na cidade especificada
```

```
    SELECT * FROM employees WHERE city = city_name;
```

```
END $$
```

```
DELIMITER ;
```

```
-- Chama o procedimento armazenado, passando o nome da cidade como parâmetro
```

```
CALL get_employees_by_city('Goiânia');
```

```
/*
```

Explicação do exemplo:

```
DELIMITER // -- : Altera o delimitador padrão de ponto e vírgula (;) para // para que o MySQL não interprete o final da criação do procedimento como o fim da instrução.
```

```
CREATE PROCEDURE get_employees_by_city(IN city_name VARCHAR(255)) -- : Cria um procedimento armazenado chamado get_employees_by_city que recebe um parâmetro de entrada (IN) chamado city_name, do tipo VARCHAR(255).
```

```
BEGIN ... END -- : Define o bloco de código do procedimento.
```

```
SELECT * FROM employees WHERE city = city_name; -- : Seleciona todos os funcionários da tabela employees onde a coluna city seja igual ao valor do parâmetro city_name.
```

```
DELIMITER ; -- : Restaura o delimitador padrão para ponto e vírgula.
```

```
CALL get_employees_by_city('Goiânia'); -- : Chama o procedimento armazenado, passando a cidade "Goiânia" como parâmetro.
```

Conclusão:

- \* Stored Procedures são uma ferramenta poderosa para gerenciar a lógica de negócio em um banco de dados MySQL, trazendo benefícios em termos de desempenho, segurança e facilidade de manutenção do código. Eles são uma parte essencial de qualquer aplicação que trabalhe com dados, permitindo que você execute operações complexas de forma eficiente e organizada.

```
*/
```

```
USE db_exemplo;
```

```
# Crie um Procedure que atualiza o preco de um curso
```

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_AtualizaPreco(NovoPreco DECIMAL(10, 2), ID INT)
```

```
BEGIN
```

```
    -- 1º) Código de atualizar o preco na tabela tb_Cursos
```

```
    UPDATE tb_Cursos
```

```
    SET Preco_Curso = NovoPreco
```

```
    WHERE ID_Curso = ID;
```

```
    -- Enviar uma mensagem de executado com sucesso.
```

```
    SELECT 'Preço atualizado com sucesso!';
```

```
END $$
```

```
DELIMITER ;
```

```

-- Atualiza SCHEMAS para aparecer em Stored Procedure
-- Selecionar arquivos da Tabela
SELECT * FROM tb_Cursos;
-- Chama a Storage Procedure
CALL sp_AtualizaPreco(432.10, 1);
-- Selecionar arquivos da Tabela
SELECT * FROM tb_Cursos;
-- ++++++
-- Usar o BD db_exemplo.
USE db_exemplo;
# Crie um Procedure que atualiza o preco de um curso
DELIMITER $$
CREATE PROCEDURE sp_AplicaDesconto(ID INT, Desconto DECIMAL(10, 2))
BEGIN
    -- 1º) Declarar variáveis importantes
    -- Variável para armazenar o preco com desconto
    DECLARE varPrecoComDesconto DECIMAL(10, 2);
    -- Variavel para armazenar nome do curso
    DECLARE varNomeCurso VARCHAR(100);
    -- Variavel para Armazenar o preco antigo do curso
    DECLARE varPrecoAntigo DECIMAL(10, 2);

    -- 2º) Atribui o alor de preco antigo á variavel varPrecoAntigo
    SET varPrecoAntigo = (SELECT Preco_Curso FROM tb_Cursos WHERE ID_Curso = ID);

    -- 3º) Atribuir o valor de Preço com desconto à variavel varPrecoDesconto
    SET varPrecoComDesconto = (SELECT Preco_Curso FROM tb_Cursos WHERE ID_Curso = ID)
    * (1 - Desconto);

    -- 4º) Atribuir o Nome do curso a variável varNomeCurso
    SET varNomeCurso = (SELECT Nome_Curso FROM tb_Cursos WHERE ID_Curso = ID);

    -- 5º) Atualizar a tabela com o novo preçopen
    UPDATE tb_Cursos
    SET Preco_Curso = varPrecoComDesconto
    WHERE ID_Curso = ID;
    -- Enviar uma mensagens de executado, só ira escrever a última mensagem
    SELECT CONCAT('Desconto de ', Desconto * 100, '% aplicado com Sucesso!') AS
'Mensagem 1 de 3';
    SELECT CONCAT('Curso ', varNomeCurso, ': Preço Antigo = R$ ', varPrecoAntigo, ',
Preço Novo = R$ ', varPrecoComDesconto) AS 'Mensagem 2 de 3';
    SELECT 'Código Executado com Sucesso!!!' AS 'Mensagem 3 de 3';
END $$
DELIMITER ;
-- Atualiza SCHEMAS para aparecer em Stored Procedure
-- Selecionar arquivos da Tabela
SELECT * FROM tb_Cursos;
-- Chama a Storage Procedure
CALL sp_AplicaDesconto(1, 0.1);
-- UPDATE tb_Cursos
-- SET Preco_Curso = 432.10
-- WHERE ID_Curso = 1;
# Alter Procedure - Alterando uma Store Procedure
-- Para alterar uma Procedure, clique com o botão direito sobe a Procedure
'sp_AplicaDesconto'
-- Opção 'Alter Stored Procedure'
-- Fazer a alteração ==> Aply ==> aply(aplicar ==> aplicar).

-- ++++++
-- Usar o BD base
USE base;
# Exerc.: Quais pedidos foram realizados na loja onde o gerente é o Marcelo
SELECT * FROM pedidos;
SELECT * FROM lojas
    WHERE Gerente = 'Marcelo Castro';

```

```

-- ++++ SubQuerye
SELECT
    *
    FROM pedidos
    WHERE ID_Loja = (
        SELECT ID_Loja
        FROM lojas
        WHERE Gerente = 'Marcelo Castro'
    );
-- ++++
-- ++++ SubQuerye com variavel
SELECT * FROM pedidos;
SELECT * FROM lojas
    WHERE Gerente = 'Julia Freitas';
-- ++++
SET @varNomeGerente = 'Julia Freitas';
SELECT
    *
    FROM pedidos
    WHERE ID_Loja = (
        SELECT ID_Loja
        FROM lojas
        WHERE Gerente = @varNomeGerente
    );
-- ++++++
-- Exerc. 2: Subquery como filtro de uma nova consulta
-- Quais produtos têm o Preco_Unit acima da Média?
SELECT AVG(Preco_Unit) FROM produtos; -- Média = 1788.1250
SELECT
    *
    FROM produtos
    WHERE Preco_Unit > (
        SELECT
            AVG(Preco_Unit)
            FROM produtos);
-- ++++++
-- Exerc. 3: Subquery como filtro de uma nova consulta
-- Quais produtos são da categoria 'Notebook'?
SELECT * FROM produtos;
SELECT
    *
    FROM produtos
    WHERE ID_Categoria = (
        SELECT
            ID_Categoria
            FROM Categorias
            WHERE Categoria = 'Notebook'
    );
-- ++++++
-- Exerc. 4: Subquery como filtro de uma nova consulta
-- Descubra todas as informações sobre o cliente que msid gerou receita para a
-- empresa?
SELECT * FROM clientes
    WHERE ID_Cliente = 38;

SELECT
    ID_Cliente,
    SUM(Receita_Venda)
    FROM pedidos
    GROUP BY ID_Cliente
    ORDER BY SUM(Receita_Venda) DESC
    LIMIT 1;

SELECT * FROM clientes
    WHERE ID_Cliente =

```

```

        (SELECT
            ID_Cliente
            FROM pedidos
            GROUP BY ID_Cliente
            ORDER BY SUM(Receita_Venda) DESC
            LIMIT 1 -- Resultado é 38
        );

-- ++++++
# 6. Subquery como filtro de uma nova consulta (Lista) - Exemplo 1
-- Exerc. 5: Subquery filtrando por meio de lista
-- Descubra qual foi a receita total associada aos produtos da marda 'DELL'?
SELECT * FROM pedidos
WHERE ID_Produto IN(1, 2, 3);

SELECT ID_Produto
FROM produtos
WHERE Marca_Produto = 'DELL';

SELECT
    SUM(Receita_Venda) AS 'Receita Marca DELL'
FROM pedidos
WHERE ID_Produto IN(
    SELECT
        ID_Produto
    FROM produtos
    WHERE Marca_Produto = 'DELL'
);

-- ++++++
# 7. Subquery como filtro de uma nova consulta (Lista) - Exemplo 2
-- Exerc. 5: Subquery filtrando por meio de lista
-- Descubra qual foi a receita total associada aos produtos da marda 'DELL'?
SELECT * FROM pedidos;
SELECT * FROM lojas;
SELECT * FROM locais;
--
SELECT * FROM locais
WHERE Região = 'Sudeste';
--
SELECT *
FROM lojas
WHERE Loja IN(
    SELECT Cidade
    FROM locais
    WHERE Região = 'Sudeste'
);
--
SELECT
    ID_Loja
FROM lojas
WHERE Loja IN(
    SELECT Cidade
    FROM locais
    WHERE Região = 'Sudeste'
);
--
SELECT *
FROM pedidos
WHERE ID_Loja IN(1, 2, 6, 7);
--
SELECT *
FROM pedidos
WHERE ID_Loja IN(
    SELECT

```

```

        ID_Loja
    FROM lojas
    WHERE Loja IN(
        SELECT Cidade
        FROM locais
        WHERE Região = 'Sudeste'
    )
);
-- ++++++
# 8. Subquery para criar uma nova coluna na consulta
-- Faça uma consulta que retorne todas as colunas da tabela Produto + uma coluna com a
média do Preço_Unit
SELECT AVG(Preço_Unit) FROM produtos;

SELECT
    *,
    (SELECT AVG(Preço_Unit) FROM produtos) AS 'Média Geral de Preço'
FROM produtos;
-- ++++++
# 9. Subquery para criar uma tabela na consulta
-- Do total de vendas por produto, qual foi a quantidade máxima vendida, mínima e
média?
SELECT * FROM pedidos;

SELECT
    MAX(Vendas) AS 'Máximo Vendas',
    MIN(Vendas) AS 'Mínimo Vendas',
    AVG(Vendas) AS 'Média Vendas'
FROM (
    SELECT
        ID_Produto,
        COUNT(*) AS 'Vendas'
    FROM pedidos
    GROUP BY ID_Produto
) AS tb;
-- ++++++
# 10. EXISTS e NOT EXISTS
/*
Em MySQL, as condições EXISTS e NOT EXISTS são usadas em conjunto com subconsultas
para verificar a existência ou não de registros que atendam a certos critérios. EXISTS
retorna TRUE se a subconsulta retornar pelo menos uma linha, indicando a existência de
registros correspondentes. NOT EXISTS retorna TRUE se a subconsulta não retornar
nenhuma linha, indicando a ausência de registros correspondentes.
EXISTS:
Verifica a existência de dados:
    * A condição EXISTS testa se uma subconsulta retorna algum resultado. Se a
subconsulta retornar pelo menos uma linha, EXISTS é avaliado como TRUE.
Ignora o conteúdo da subconsulta:
    * O conteúdo da subconsulta não é importante para a condição EXISTS. O que
importa é se a subconsulta retorna algo ou não.

Pode ser usado em SELECT, INSERT, UPDATE, e DELETE:
EXISTS pode ser utilizado em várias instruções SQL para filtrar dados ou realizar
ações com base na existência de registros.
Exemplo:
*/
SELECT * FROM clientes
WHERE EXISTS (SELECT 1 FROM pedidos WHERE pedidos.cliente_id = clientes.id);
/*
Este exemplo retorna todos os clientes que possuem pelo menos um pedido associado.
NOT EXISTS:
    * Verifica a não existência de dados: A condição NOT EXISTS é o oposto de
EXISTS. Retorna TRUE se a subconsulta não retornar nenhuma linha.
    * Funciona como o inverso de EXISTS: Se EXISTS retorna TRUE, NOT EXISTS retorna
FALSE, e vice-versa.

```

Pode ser usado em SELECT, INSERT, UPDATE, e DELETE: Semelhante a EXISTS, NOT EXISTS pode ser usado em várias instruções SQL.

Exemplo:

```
*/
SELECT * FROM produtos
WHERE NOT EXISTS (SELECT 1 FROM vendas WHERE vendas.produto_id = produtos.id);
/*
```

Este exemplo retorna todos os produtos que não foram vendidos.

Diferenças entre NOT IN e NOT EXISTS:

- \* NOT IN é usado para comparar valores diretamente, enquanto NOT EXISTS é mais adequado para verificar a existência de registros em subconsultas relacionadas.

- \* NOT EXISTS pode ter melhor desempenho em subconsultas com muitos registros e também permite o uso de índices, enquanto NOT IN pode ter problemas com valores NULL.

- \* Em geral, NOT EXISTS é recomendado para subconsultas, e NOT IN é mais adequado quando você já possui os valores como parâmetro.

```
*/
-- ++++++
USE base;
# Exerc.: Você deverá verificar se todas as categorias possuem pelo menos 1 exemplar
de produtos (na tabela de produtos)
-- caso alguma categoria não possua nenhum exemplar você deverá descobrir qual é ou
quais são.
SELECT * FROM categorias;
SELECT * FROM produtos;
--
SELECT
    ID_Categoria
FROM categorias;
--
SELECT
    ID_Categoria
FROM produtos;
-- Que existem ( 1, 2, 3, 4, 5 e 6)
SELECT
    ID_Categoria
FROM categorias
WHERE EXISTS (
    SELECT
        ID_Categoria
    FROM produtos
    WHERE categorias.ID_Categoria = produtos.ID_Categoria
);
-- Que NÃO existe(7)
SELECT
    ID_Categoria
FROM categorias
WHERE NOT EXISTS (
    SELECT
        ID_Categoria
    FROM produtos
    WHERE categorias.ID_Categoria = produtos.ID_Categoria
);
-- Que NÃO existe(7)
SELECT
    *
FROM categorias
WHERE NOT EXISTS (
    SELECT
        *
    FROM produtos
    WHERE categorias.ID_Categoria = produtos.ID_Categoria
);
```